



**Politecnico
di Torino**

**Rakuten
Mobile**

Design and implementation of an eBPF-based Runtime Security Monitoring Tool

Master's Degree in Cybersecurity Engineering

Supervisor:

Prof. Riccardo Sisto

Company Supervisor:

Dr. David Soldani

Academic Year
2025-2026

Simone Romano s344024 · Giuseppe Insalaco s343410

Presentation Outline

COMMON FOUNDATION

01 | Project context and design constraints

02 | Common architecture

INDIVIDUAL CONTRIBUTIONS

03 | Host monitoring

SIMONE ROMANO

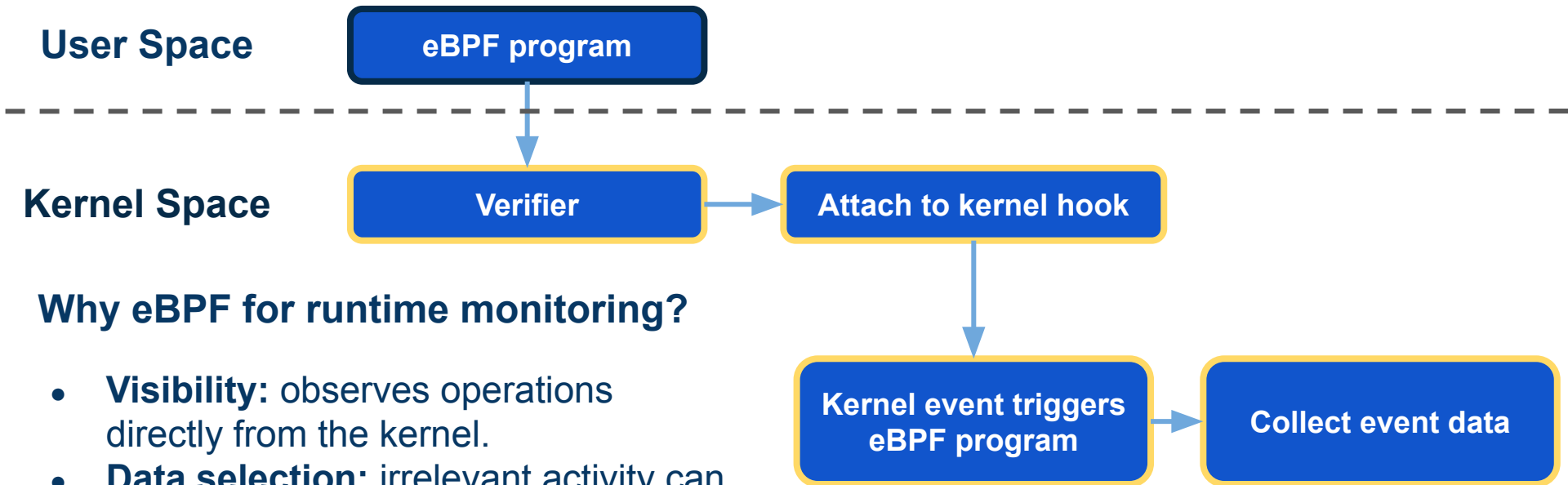
04 | Network monitoring

GIUSEPPE INSALACO

CLOSING

05 | Overall conclusions

What is eBPF?



Why eBPF for runtime monitoring?

- **Visibility:** observes operations directly from the kernel.
- **Data selection:** irrelevant activity can be discarded before transfer to userspace.
- **Safe execution:** verifier checks programs before loading.

Project Motivation and Objective

MOTIVATION

A monitoring system tailored to specific requirements

The company required an internally owned and maintained solution, tailored to specific requirements, without depending on an existing general-purpose runtime security tool.

OBJECTIVE

Design, implement and evaluate an eBPF-based runtime security monitoring tool

Collect security-relevant activity directly from the Linux kernel and process it in user space to produce normalized events that can be analyzed by a security analyst.

Project Requirements and Architectural Reference

01 PLATFORM COMPATIBILITY

Rocky Linux 8.10

4.18-based enterprise kernel

Program types, helpers, maps and attachment mechanisms had to be verified on the deployed kernel.

02 OPERATIONAL REQUIREMENT

CPU usage ~5%

of one logical core

Applied to the complete end-to-end system, including kernel collection and user-space processing

03 ARCHITECTURAL REFERENCE

Tracee

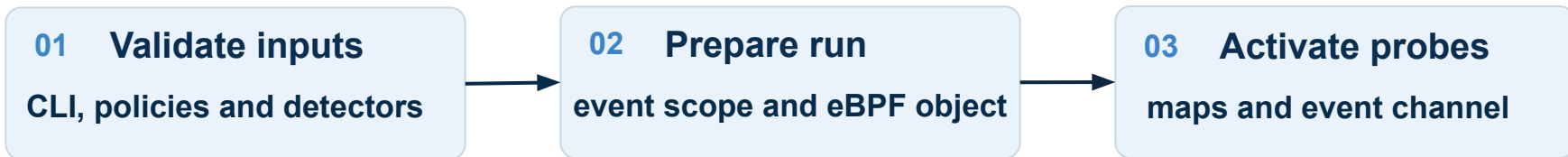
Open-source multi-kernel runtime security and observability tool

Reference for the common event model and the separation between kernel collection and user-space processing.

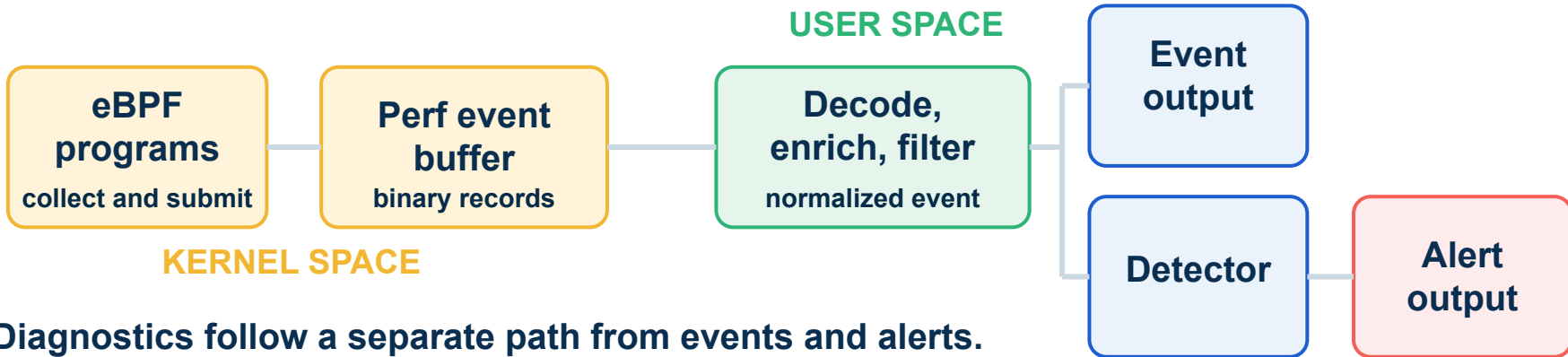
End-to-End Monitoring Pipeline

Setup prepares the instrumentation. Event processing begins after the probes are attached.

SETUP PATH · USER SPACE



EVENT PATH



Diagnostics follow a separate path from events and alerts.

Binary Contract and Event Normalization

Kernel
[event_context_t + argnum + args buffer]

From binary records

Shared C/Go Registry

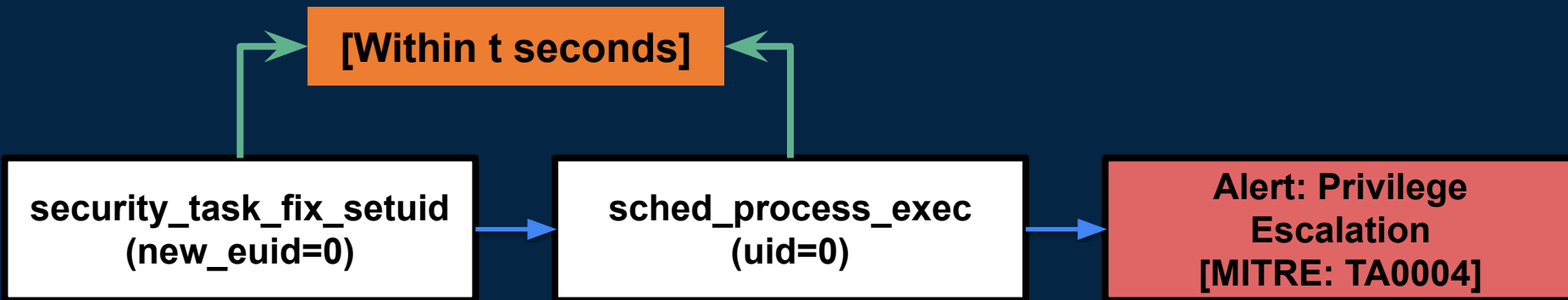
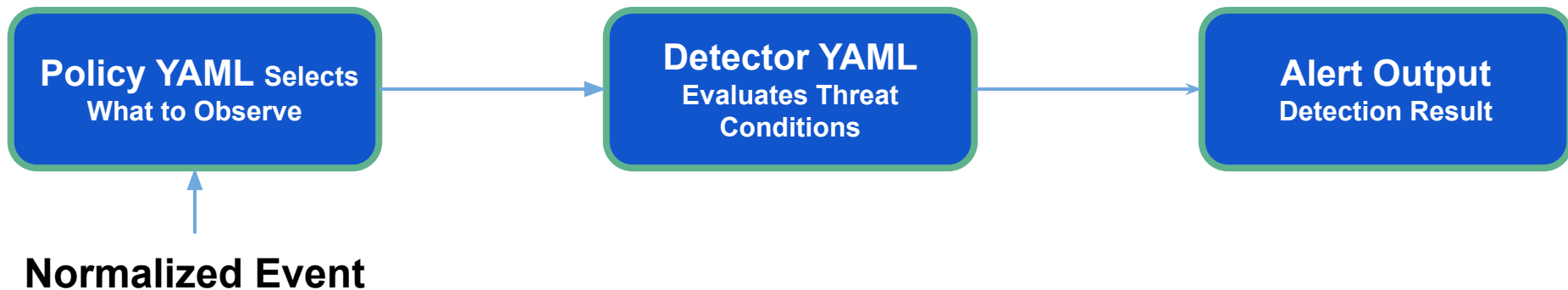
to typed, indexed arguments

Userspace

Event:	security_file_open
Path:	/etc/shadow
Flag:	O_RDONLY

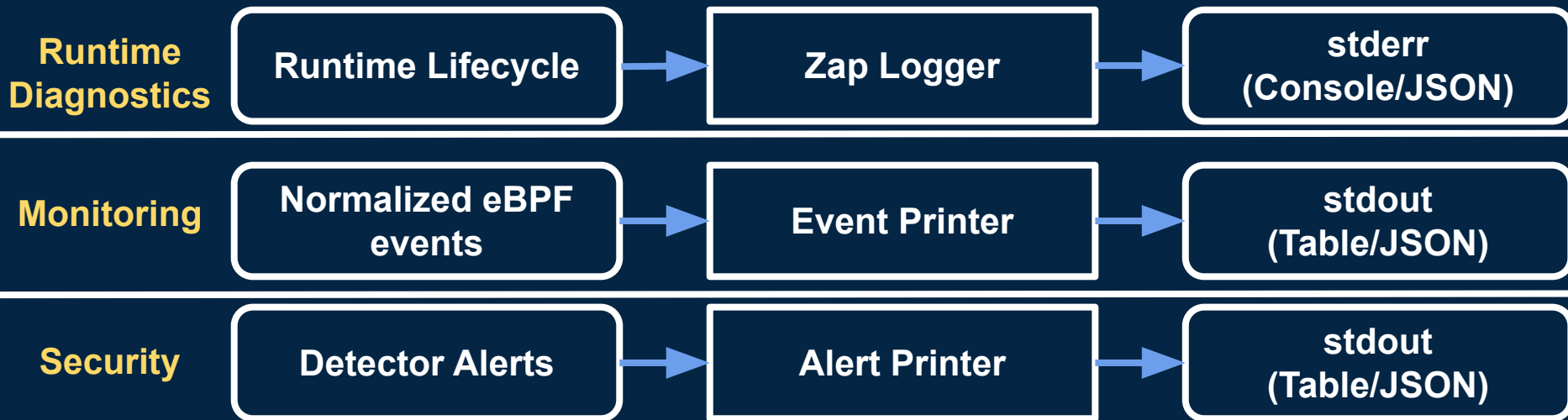
- **Binary contract:** the information must be transferred to Go user space in a consistent format.
- **Decoding schema:** tells the Go userspace the event name, the argument names and their types.
- **Enrichment:** Userspace normalization of capabilities, flags, bitmasks, and errno values.

Policy and Detection Layer (YAML)



Configuration, Logging, and Output Routing

- **Clear separation:** Zap handles runtime lifecycle logs, while dedicated printers format events and alerts.
- **--alerts-only mode:** Hides raw terminal output while keeping events available to detectors.
- **Kubernetes readiness:** Independent streams support containerized and centralized deployments.



Experimental Validation Setup

Monitor Pod

eBPF Agent

Filtering on UID
10001

Trigger Pod

Exploit Client
(Attacker)

Target Pod

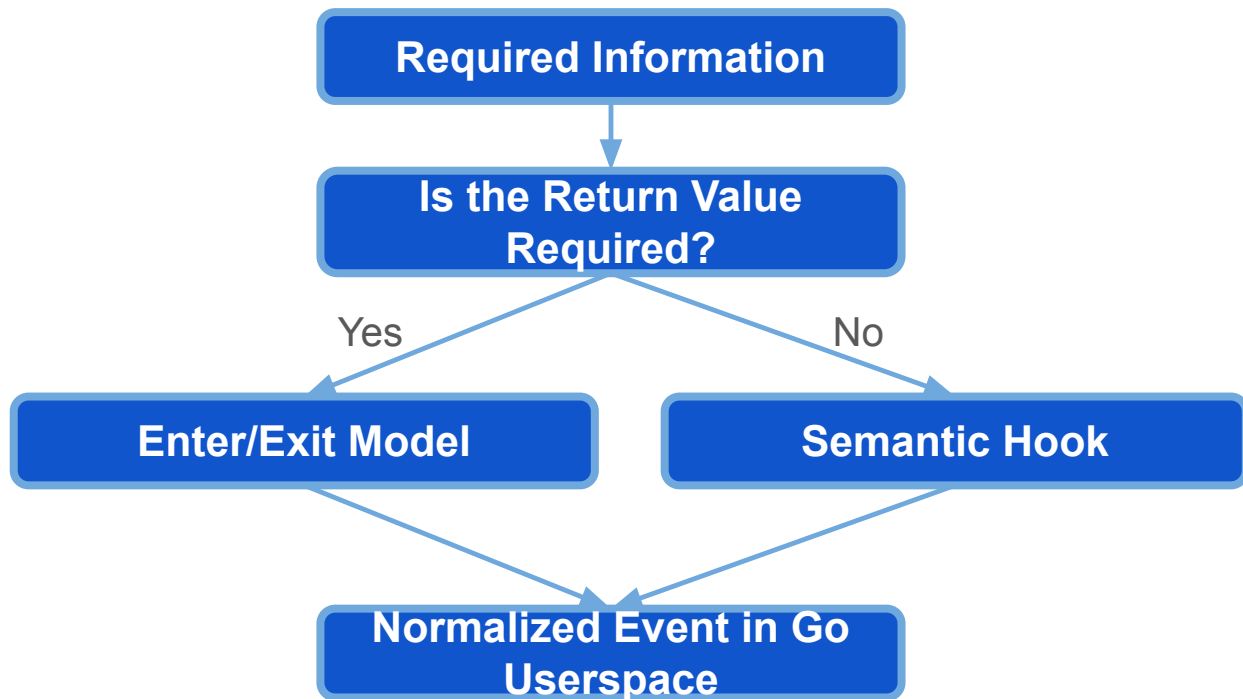
Vulnerable
Service
(UID 10001)



Shared Linux Kernel (Rocky Linux 8.10)

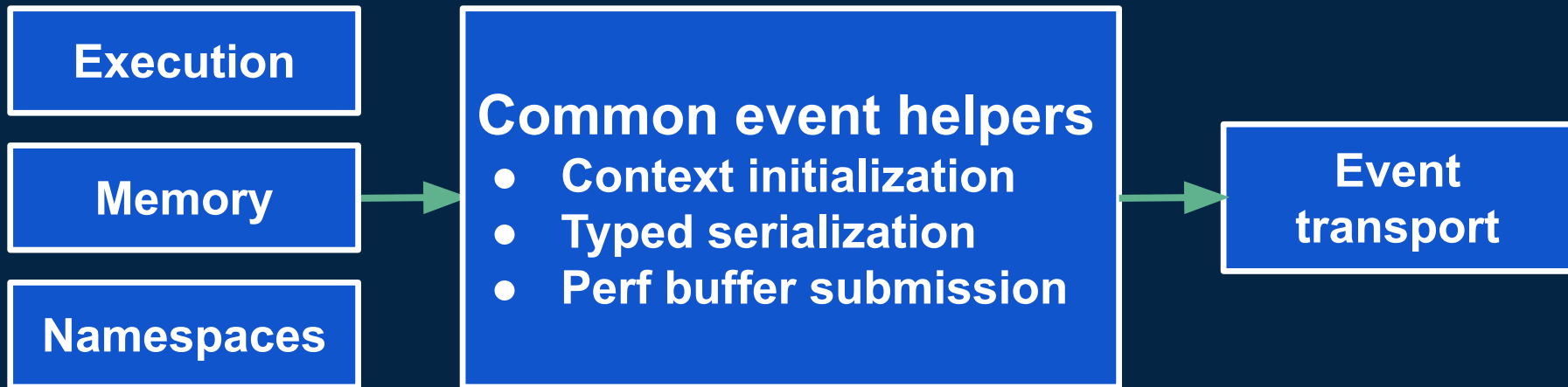
Host level Hook Implementation

- **Main challenge:** Host operation rarely provides the complete result to identify an attack
- **Key principle:** Each event uses the hook that provides the required evidence.
- **Enter/exit model:** Entry stores arguments; exit adds the final return value.



Common eBPF Implementation Pattern

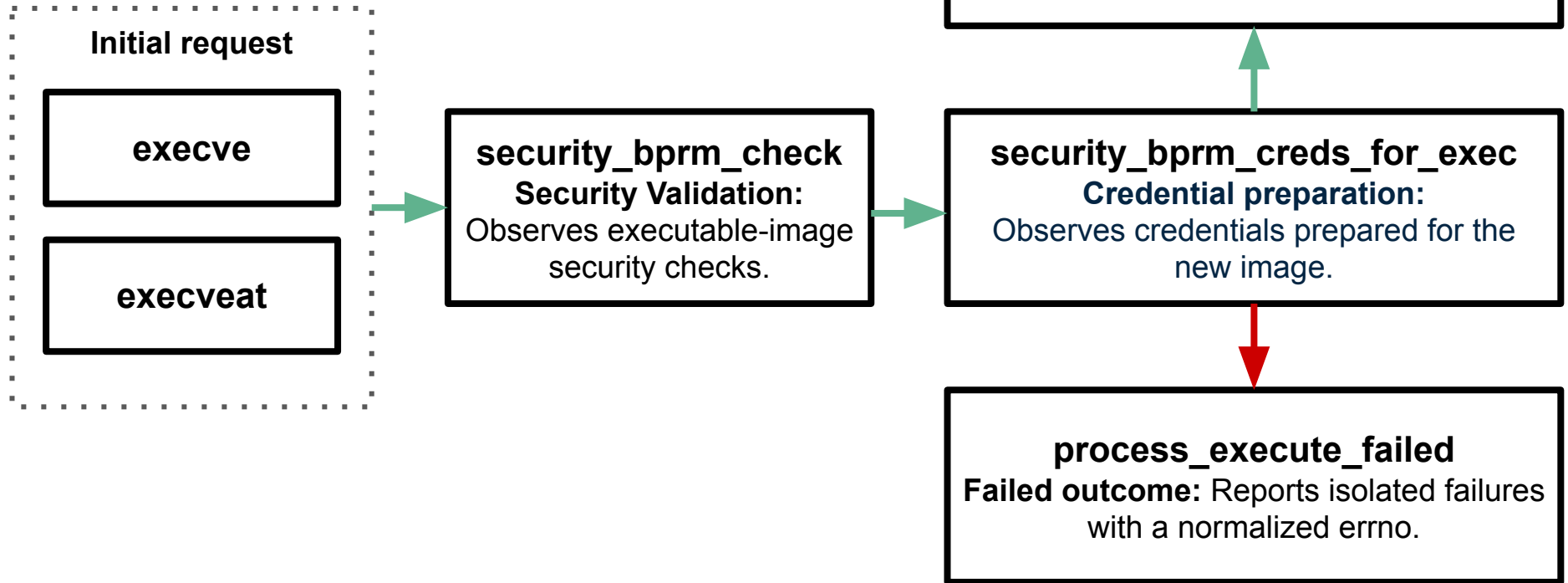
Different context



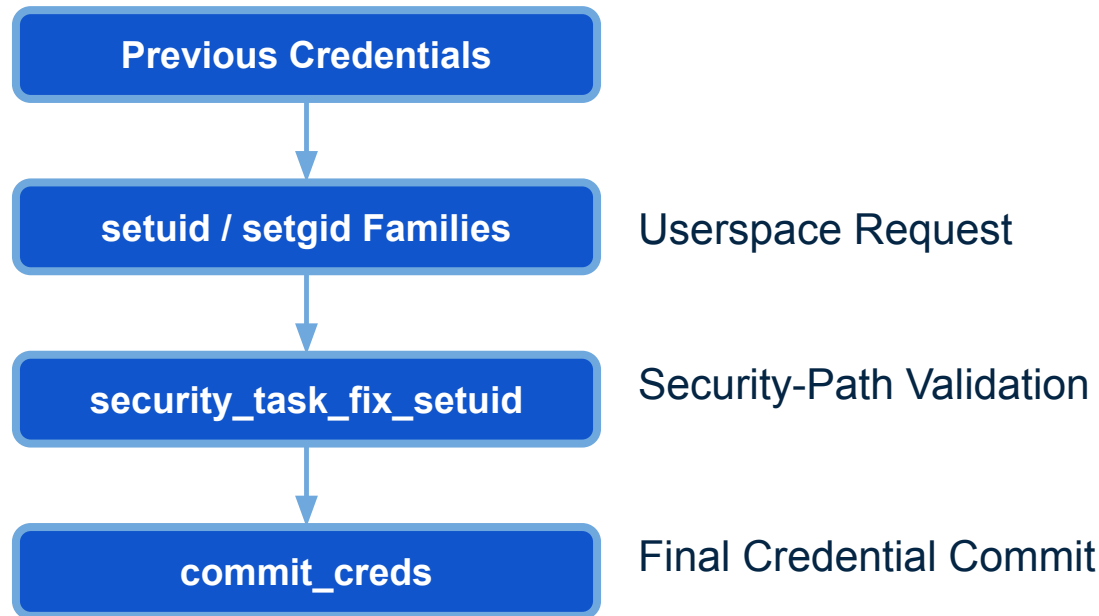
Only operation specific data changes

One bounded event format for all host-level hooks

Process Execution Monitoring: Attempts and Outcomes



Identity, Credentials, and Capabilities



Capability and process control

- **cap_capable:** Observes checks for specific capabilities.
- **ptrace:** Observe direct process interaction and state manipulation.

File, Permission, and Filesystem Security

File Access

- **open**: reports the syscall outcome.
- **security_file_open** / **security_file_permission**: expose stable metadata (path, inode, device, ecc..)

Permission and ownership

- The **chmod** and **chown** families monitor permission and ownership changes, including return values.

Filesystem changes

- **unlink**, **rename**, **symlink** and **mknod**: track structural filesystem changes.

Mounting

- **security_sb_mount** / **security_sb_unmount**: observe mount points and associated flags.

Memory and Namespaces Monitoring

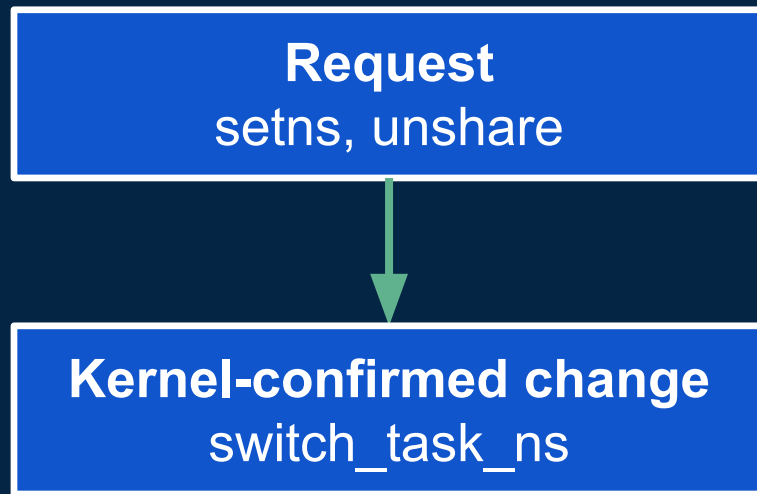
Memory

Security relevant activity can modify or access process memory without creating files or starting new processes

- **Memory mappings:**
mmap, mprotect, pkey_mprotect
- **Memory-backed files:**
memfd_create.
- **Cross-process access:**
process_vm_readv, process_vm_writev

Namespaces

Define process isolation in containerized environment. Hooks track movement in isolation boundaries



AES beta exploit and detected evidence

Target Service

**Vulnerable CTF
encryption service**

**Kubernetes pod, UID
10001**

Flag:
`/opt/aes-beta/flag`

Final payload:
`system("cat flag")`

Kernel Evidence

/bin/sh executed
`sched_process_exec`

/bin/cat executed
`sched_process_exec`

**Flag access
observed**
`security_file_open`

Successful open
`open -> fd 2`

Detector Result

High severity alert

**Command execution
and flag access**

4 correlated events

MITRE ATT&CK:

TA0002 Execution

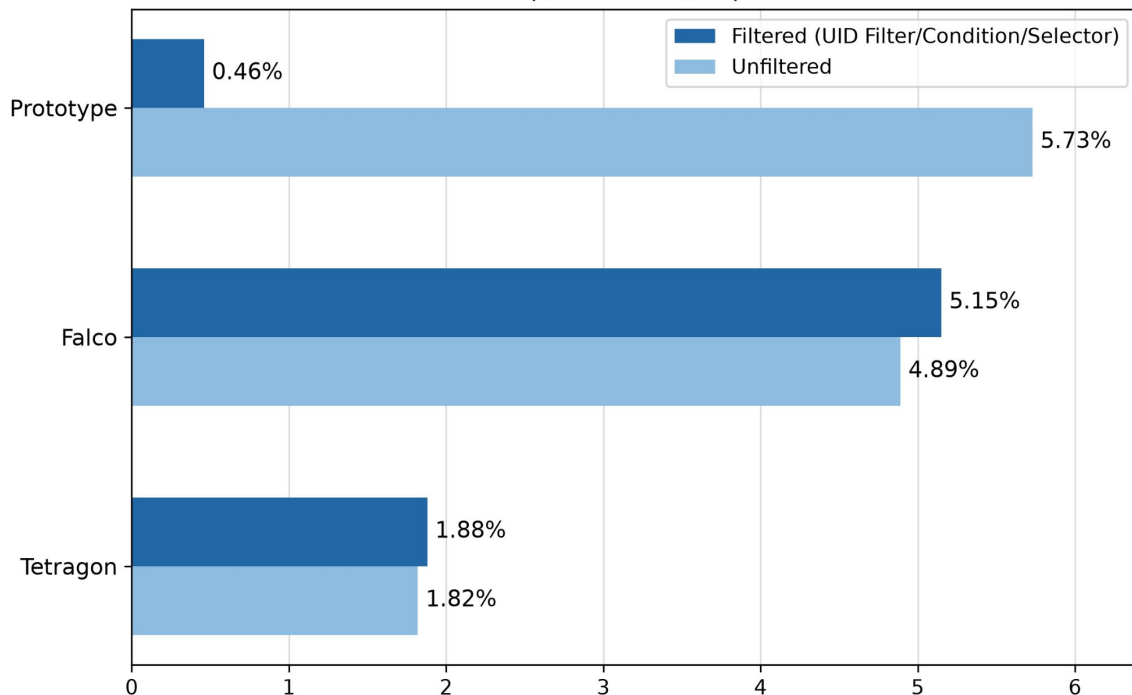
TA0009 Collection

T1059.004 Unix shell

T1005 Data from Local
system

Evaluation Results: CPU Consumption

Mean Monitor-Container CPU Consumption
(% of one core)



- All retained runs recovered the flags and produced the required evidence.
- The prototype reduced mean CPU by about 92% through filtering before transport.
- Falco and Tetragon apply **UID** selection at different stages of their execution flow.

Networking Subsystem

The networking subsystem builds context-rich events to support the analysis of deviations from the expected network profile.

01

OBSERVE

Socket operations
and packets

02

CORRELATE

Process context
with each packet

03

DECODE

Binary records
into
normalized events

04

ENRICH

Socket fields and
protocol metadata

Kernel space: observation and correlation

User space: decoding and enrichment

Network Observation Surfaces

Socket-level kprobes observe socket operations. cgroup_skb programs collect packet traffic.

SOCKET OPERATIONS

LSM socket dispatch functions

kprobes on `security_socket_*` functions

Available context

current task and native socket arguments

Collected state

public socket events and process/workload context

Platform constraint

kernel 4.18 lacks the BPF LSM program type

PACKET TRAFFIC

cgroup_skb packet programs

attached to a directory in the cgroup v2 hierarchy

Attachment

cgroup v2 directory with ingress and egress attach types

Available context

packet available through the restricted BPF context:
`struct __sk_buff`

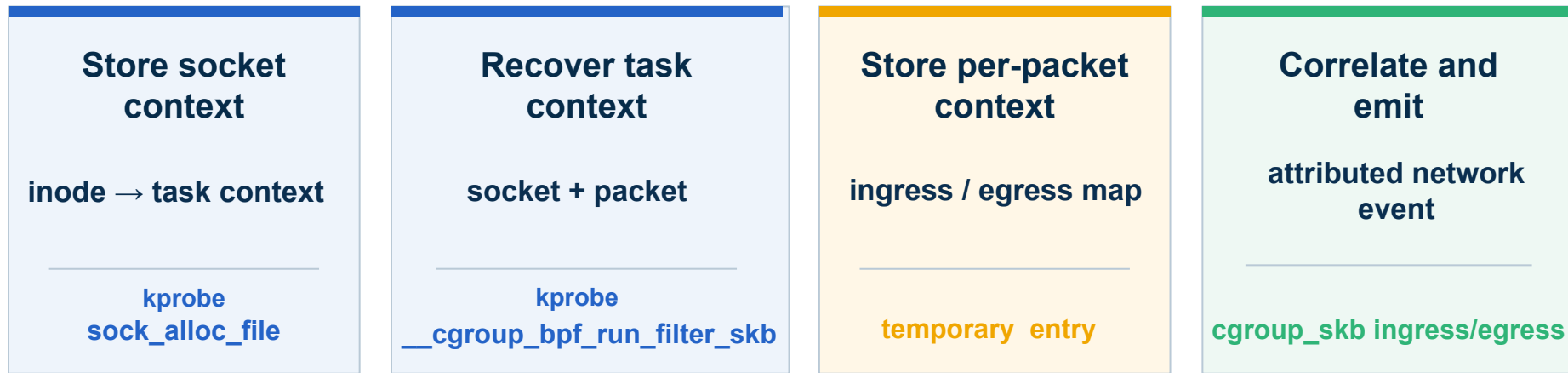
Collection state

packet headers and payload bytes, but not process identity

Socket probes provide process identity. cgroup_skb programs provide packet visibility.

Process-Packet Correlation

Socket-associated task context reaches the packet program through a short-lived handoff.



SHARED HANDOFF KEY

Timestamp + IP packet length + L4 protocol + source IP + destination IP + checksum

No current-task fallback

If attribution fails, the monitor emits no event and lets the packet continue unchanged.

Kernel-side Packet Parsing and Event Fan-out



How each layer is identified

L3: Socket family and IP version
IP protocol dispatch: IP protocol fields
L7: Port numbers + payload signatures

One packet may generate multiple layer-specific events

net_packet_raw

net_packet_ip

net_packet_tcp

net_packet_udp

net_packet_icmp

net_packet_dns

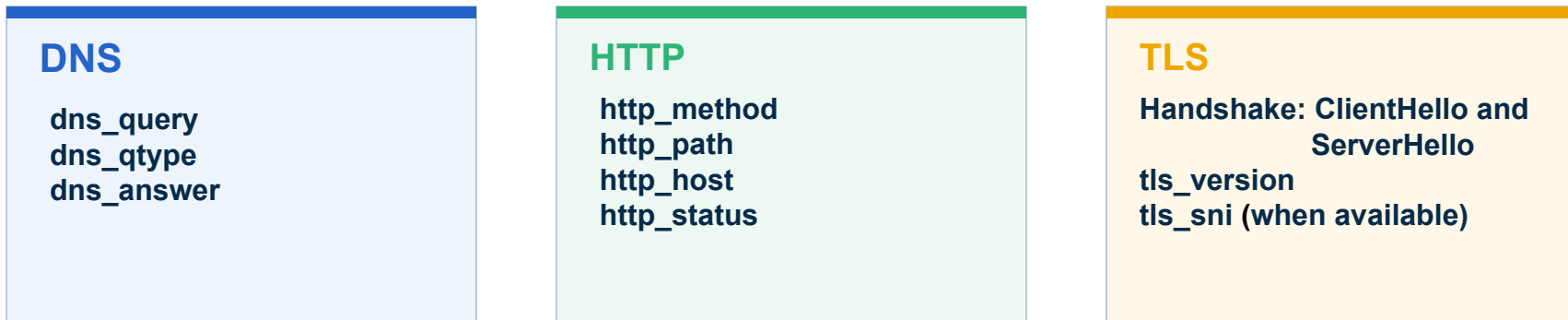
net_packet_http

net_packet_tls

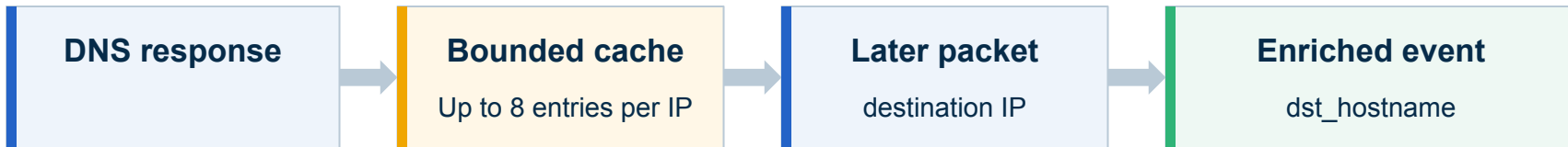
Benefit: layer-specific events simplify filtering, protocol decoding and detector rules.

User-Space Protocol Decoding and Enrichment

Selected packet bytes become structured arguments for policies, detectors and output components.



DNS CONTEXT FOR LATER EVENTS



Context, not attribution

The cached hostname provides context for the destination IP, but does not prove that the same process performed the DNS lookup.

Functional Evaluation

EXPERIMENTAL SETUP

Single-node Minikube cluster

CROSS-POD VISIBILITY

Events retained their originating workload and process context.

ATTACK SCENARIO

- | | | |
|----|-------------------|---|
| 01 | SSRF | service → attacker :9000 (HTTP GET) |
| 02 | Command injection | reverse shell: service → attacker :4444 |
| 03 | Data exfiltration | service → attacker :9001 (HTTP POST) |

BEHAVIOR-BASED DETECTORS

- | | | |
|----|------------------------|----------|
| D1 | Unexpected egress port | 3 alerts |
| D2 | Outbound HTTP upload | 1 alert |

DETECTION RESULTS

BENIGN TRACE

0 alerts

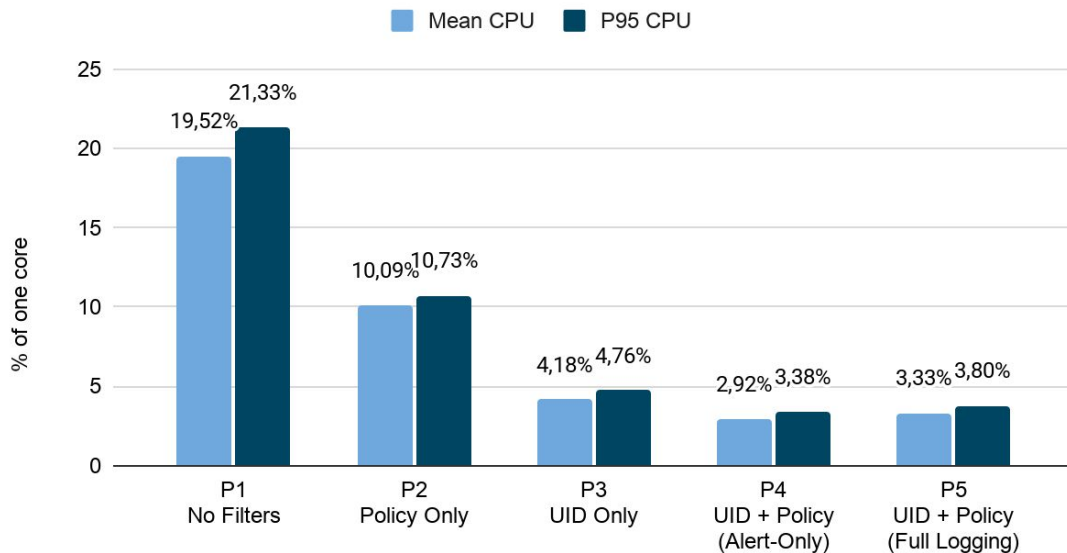
CONTROLLED ATTACK

4 alerts

Detection relied on network-profile deviations, not exploit-specific signatures.

Performance Evaluation

CPU Consumption by Configuration Profile



TEST CONDITIONS

120 s

measurement window

10 ms

between request cycles

~ 1 s

CPU sampling interval

KEY FINDINGS

71 - 79%

lower mean CPU with
UID filtering

**<5% CPU
usage**

for every UID-filtered
profile

Conclusions

- 01 | End-to-end monitoring was successfully implemented and validated on the target Rocky Linux 8.10 platform.
- 02 | A common event model integrated host and network observations into the same processing and detection pipeline.
- 03 | The functional evaluations produced the expected evidence and alerts in both attack scenarios, with no alerts for the evaluated benign traces.
- 04 | Early workload-scoped filtering was essential: all UID-filtered configurations remained below the 5% CPU requirement.

Thank you for the attention!